UI extensibility

UI extensions overview

Learn about the different types of UI extensions and how to build them.

HubSpot UI extensions allow you to build rich, contextual custom interfaces in HubSpot. Tailor CRM records, workflows, and app experiences with dynamic data, custom layouts, and interactive React-based components, all powered by the **UI extensions SDK**.

Below, learn more about when to use UI extensions, how they work, and how to get started.

> If you have an existing legacy CRM card, check out HubSpot's **Legacy CRM Card to UI Extension Converter**, which provides an example implementation of an app card that functions like a legacy CRM card.

When to use UI extensions

UI extensions are ideal when:

- You want to display external system data in HubSpot.
- You need to guide users through multi-step flows.
- You want to enable users to customize CRM records with richer functionality than HubSpot's default UI allows.
- You want HubSpot to host your app's configuration UI.
- You need a secure and scalable way to embed custom UI logic without hosting your own frontend architecture.

How UI extensions work

At a high level, a UI extension consists of:

A React component (`.jsx` or `.tsx`) that defines your UI. TypeScript is fully supported and recommended for better type safety.

Optional data fetching via the `hubspot.fetch()` API to request data from external services.

During runtime:

1. HubSpot loads your extension in a secure, sandboxed environment.
2. Context is passed into your component, including information about the current user, HubSpot account, and (for CRM extensions) the current record's ID and object type.
3. Your component can fetch CRM data using **SDK hooks** or call external services using `hubspot.fetch()` .
4. The UI renders and reacts to user interactions.

This architecture ensures secure isolation, predictable performance, and a consistent developer experience.

UI extensions are subject to the following limitations:

Each `hubspot.fetch()` request has a maximum timeout of 15 seconds.

Request and response payloads are limited to 1 MB each.

Each app is allowed up to 20 concurrent `hubspot.fetch()` requests per account.

`hubspot.fetch()` does not support custom request or response headers (except for the `Authorization` header).

What you can build

App cards

Create app cards to surface insights, trigger external workflows, or bring data from third-party services directly into HubSpot. You can build app cards for **CRM records**, **preview panels**, and the **help desk** sidebar.

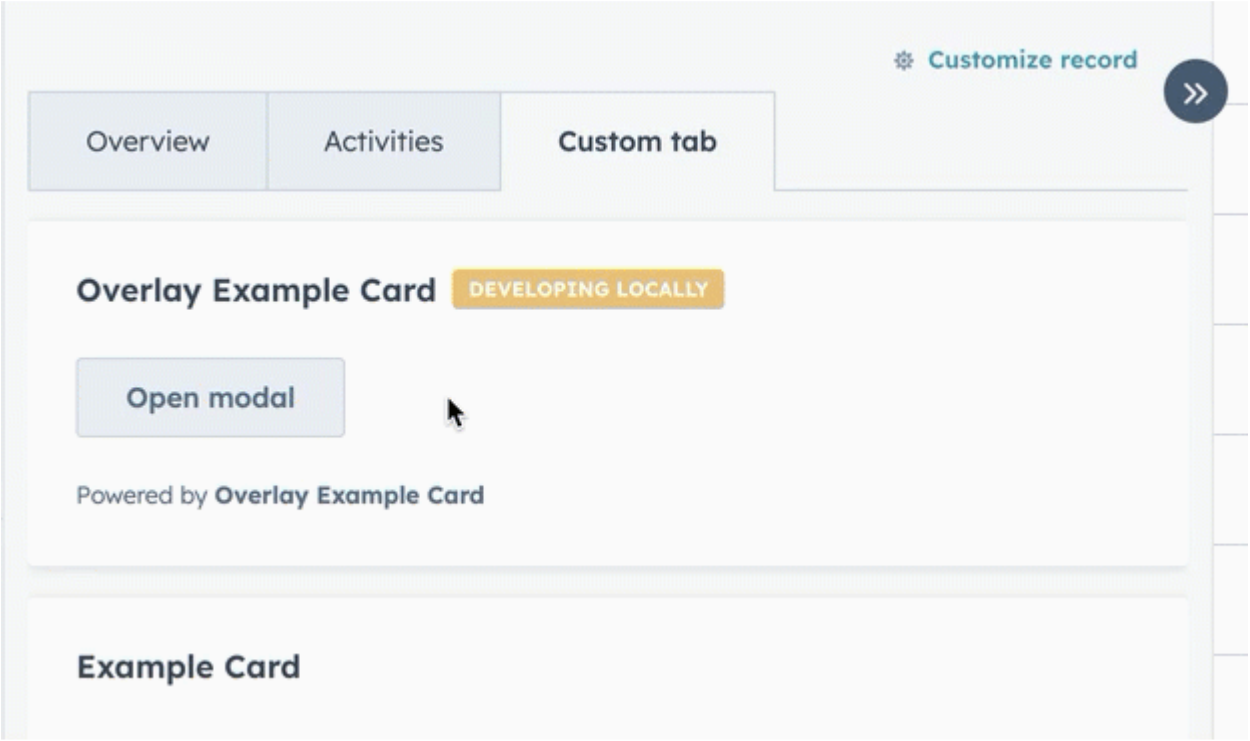
HubSpot Developers

›

Triggering external workflows or actions from within a record.

Rendering custom layouts, accordions, tables, and charts.

Learn more about **building app cards**.



App home pages

A home page is a full-screen extension, ideal for dashboards, analytics views, and complex workflows.

Common use cases include:

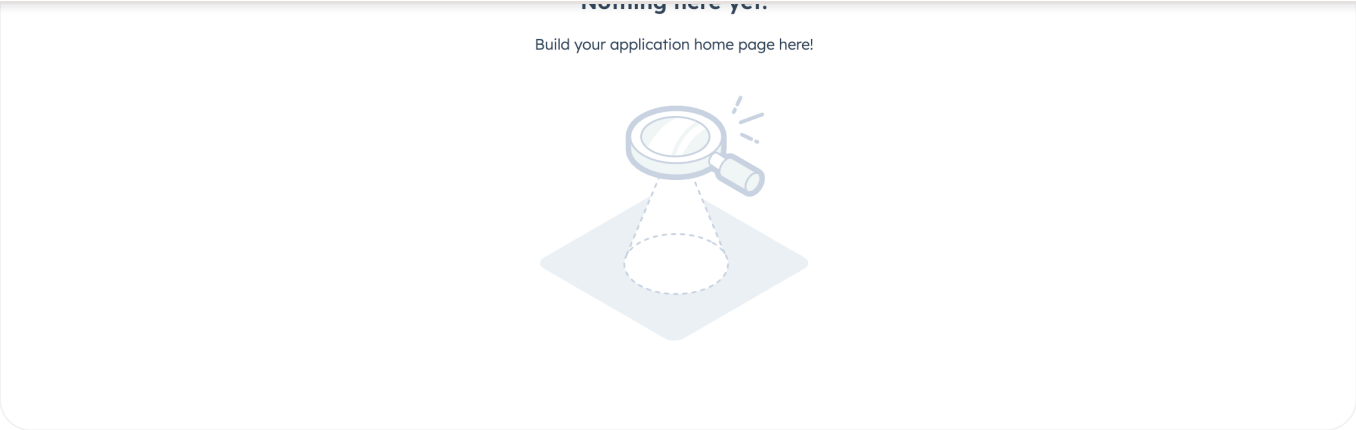
Building analytics dashboards.

Enabling multi-step flows or guided processes.

Aggregating cross-object CRM insights.

Creating custom workspace experiences.

Learn more about **building app home pages**.



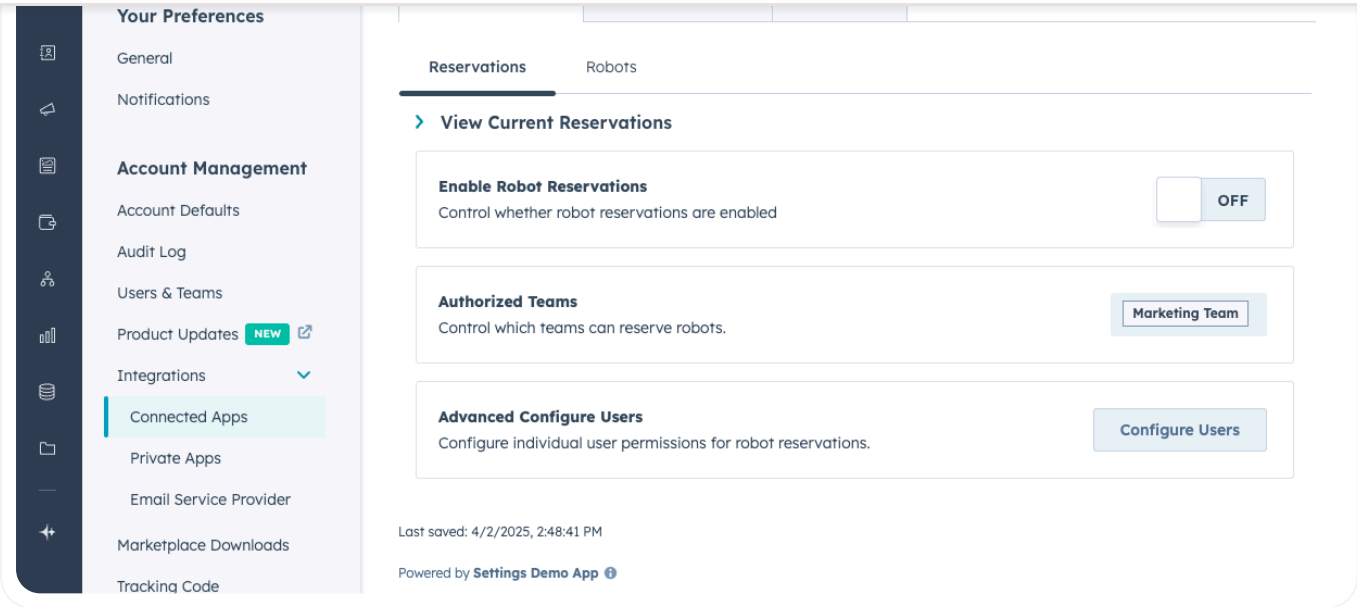
Settings pages

Settings pages allow you to build configuration UIs that live inside HubSpot's settings. These pages are ideal for:

- Managing API keys or OAuth credentials.
- Managing feature toggles or custom configuration.
- Connecting external services.
- Mapping CRM properties to external systems.

Learn more about **building app settings pages**.

HubSpot Developers



Building UI extensions

All UI extensions share the same development workflow:

1. **Create the extension**: run the `hs project add` command to add a card, home page, or settings page component to an existing project. Or, follow the **quickstart** to build your first extension.
2. **Build with React**: build your extension's UI with React or TypeScript, using HubSpot's **UI component library** to render elements.
3. **Register with the SDK**: use `hubspot.extend()` to register your extension with HubSpot.
4. **Upload and test**: run `hs project upload` to deploy, and `hs project dev` for local development. You can also build testing into your project using a set of provided **testing utilities**.

Supported locations

UI extensions can be built for specific locations throughout HubSpot. When configuring an extension, you'll specify a `location` value in the extension's JSON configuration file.

HubSpot Developers

```
    "name": "Hello Example App",
    "description": "A description of the card's purpose.",
    "location": "crm.record.tab",
    "entrypoint": "/app/cards/ExampleCard.jsx",
    "objectTypes": ["contacts"]
  }
}
```

Location	Value	Description
CRM record middle column	`crm.record.tab`	Appears in the middle column of CRM record pages, within default or custom tabs. Also available in the sales workspace target accounts preview panel when `objectTypes` is `COMPANIES`.
CRM record sidebar	`crm.record.sidebar`	Appears in the right sidebar of CRM record pages. Also available in the sales workspace deals sidebar when `objectTypes` is `DEALS`.
CRM preview panel	`crm.preview`	Appears in the preview panel accessible throughout the CRM, including record pages, index pages, board views, and the lists tool.
Help desk sidebar	`helpdesk.sidebar`	Appears in ticket sidebars within help desk, including the ticket preview panel and the right sidebar of ticket views.
App home page	`home`	Appears in the .
App settings page	`settings`	Appears in the .

Learn more about supported objects and location configuration in the .

The UI extensions SDK

The provides the foundation for all UI extensions, including:

Context access: retrieve information about the current user, account, and CRM record.

Fetching external data

Use the `hubspot.fetch()` API to request data from external services. Learn more about **fetching data for UI extensions**, including:

- Configuring permitted URLs.
- Request signing and validation.
- Proxying requests during local development.

UI components

HubSpot provides a library of reusable components for building extension interfaces, including:

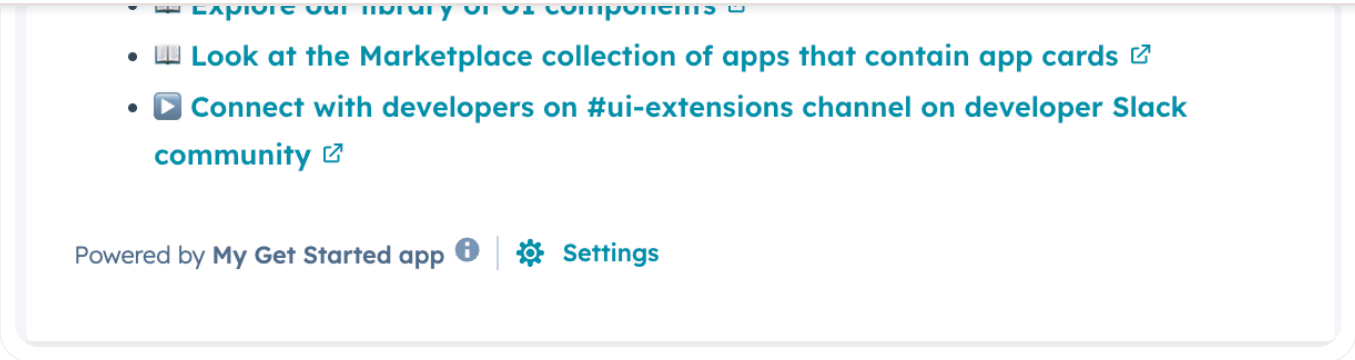
- **Standard components**: general-purpose components like buttons, forms, tables, and layout components. Imported from `@hubspot/ui-extensions` .
- **CRM data components**: components that automatically fetch and display data from CRM records, such as property lists, association tables, and reports. Imported from `@hubspot/ui-extensions/crm` .
- **CRM action components**: components that trigger built-in CRM actions like adding notes, sending emails, or creating records. Imported from `@hubspot/ui-extensions/crm` .

> Check out HubSpot's **Figma Design Kit** to help you create more detailed designs, share demos with stakeholders, and keep teams aligned during the development process.

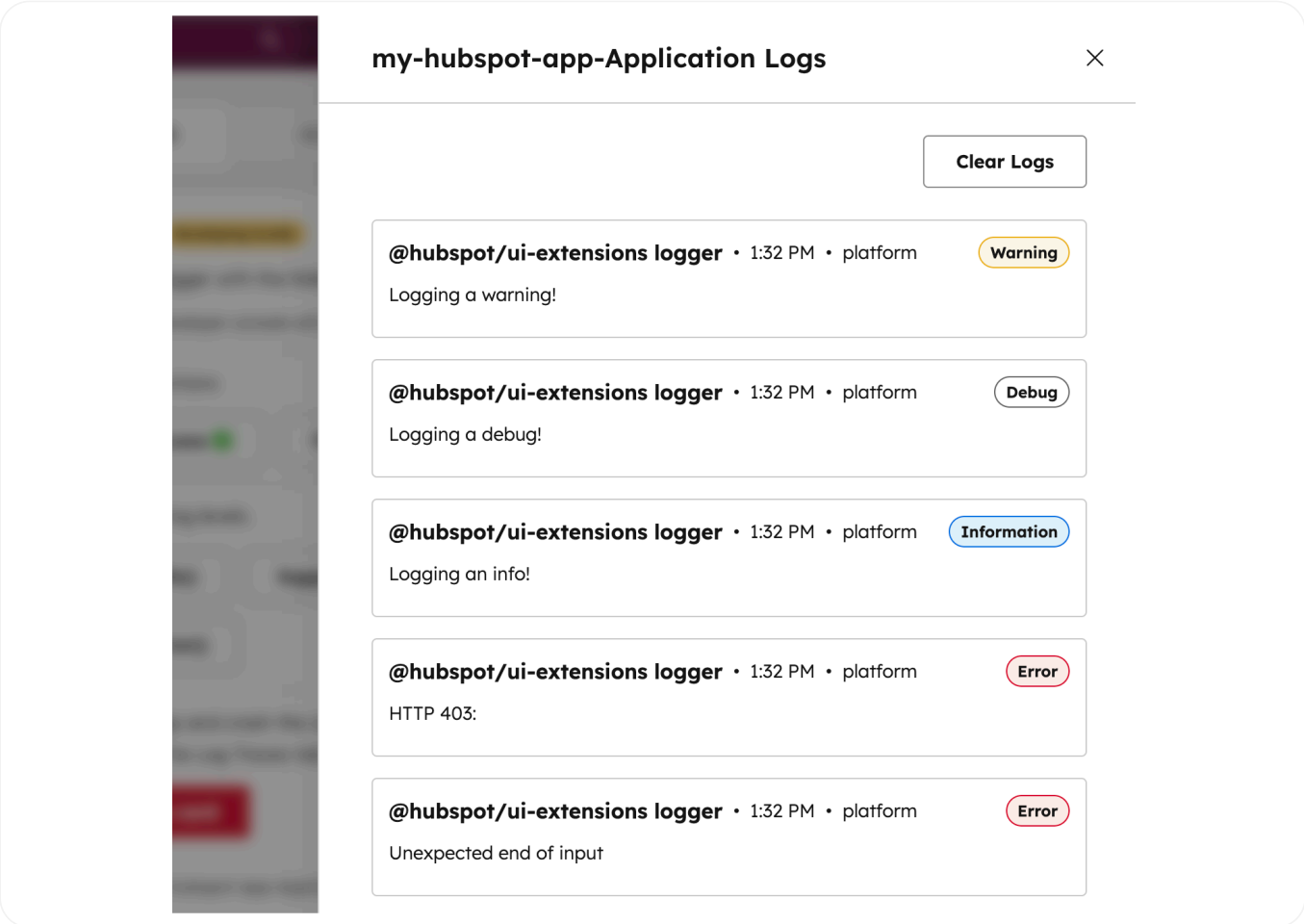
Local development and testing

Use the `hs project dev` command to start a local development server with hot reload. This enables you to preview changes to your extension in real-time without needing to upload your project after every change.

HubSpot Developers



To help debug during local development, HubSpot provides an extension logs panel that surfaces errors, warnings, and custom log messages directly in the browser.



For deployed extensions, you can review render logs, `hubspot.fetch()` requests, and custom log output. Learn more about **logging and monitoring for UI extensions**.

The UI extensions SDK also provides **testing utilities** for writing unit tests, including:

Debugging rendered component trees.

Code quality and linting

HubSpot provides an ESLint configuration specifically designed for UI extensions. This catches common issues like using unavailable browser APIs and incorrect imports before runtime. Learn more about **linting for UI extensions**.

Next steps

Quickstart: create your first app on the developer platform.

Create an app card: create a custom card that displays on CRM records.

Create a home page: build a landing page for your app.

Create a settings page: add a configuration interface for your app.

Last modified on February 19, 2026

Was this page helpful? Yes No

Integrate AI tools with the HubSpot MCP server (BETA)
‹ Previous

App cards overview
Next ›